Actividad 9: Práctica en Papel
«Pensar antes de programar»

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López

Curso: Laboratorio de Programación — 5.º año 3.ª división (Grupos A-B)

Profesor: Mansilla Muñoz York Elías

Fecha: 11/11/2026

Introducción

La Actividad 9 de Laboratorio de Programación, titulada «Práctica en papel: pensar antes de programar», tiene como objetivo pedagógico desacoplar el razonamiento lógico del lenguaje de programación. En lugar de comenzar escribiendo código, cada problema se resuelve primero en la carpeta de la materia mediante dos herramientas de análisis: el diagrama de flujo (dibujado con la aplicación de color indicada para cada ejercicio) y la tabla de prueba de escritorio, que permite trazar a mano el valor de las variables paso a paso, tal como lo haría el procesador.

La metodología de trabajo siguió ese orden de forma estricta. Los cuatro ejercicios propuestos —el sensor de aprobación, el bucle del contador de vidas, el registro de jugador en memoria y el control de inventario con arreglos— se resolvieron manualmente con lápiz y papel. Luego, la lógica validada a mano se tradujo a un código de referencia en C++ (archivo src/main.cpp), versionado de forma individual en la rama algoritmica del repositorio Git del estudiante, respetando la estructura obligatoria de carpetas de la consigna. El programa se compiló con el compilador g++ y se ejecutó con los datos de prueba oficiales para contrastar la salida real contra la salida predicha en papel, dejando constancia en la captura matriz_ejecucion.png.

Este informe se organiza según la Plantilla APA v7 provista en clases: portada, introducción, desarrollo, conclusión y referencias. El desarrollo explica la solución técnica de cada ejercicio, presenta las tablas de prueba de escritorio resueltas, la captura de la matriz de ejecución en consola y los espacios reservados para las fotografías de las cuatro hojas manuscritas de la carpeta, que documentan el trabajo realizado a mano. En las referencias se adjunta la URL pública del repositorio.

Desarrollo

Metodología de resolución

Cada ejercicio se abordó con la secuencia «pensar → dibujar → trazar → programar → verificar». Primero se analizó el enunciado y se identificaron las variables, constantes y estructuras de control involucradas. Luego se dibujó el diagrama de flujo en la hoja de carpeta con su color indicado (verde, amarillo, naranja o rojo), utilizando la simbología estándar: óvalo para inicio/fin, paralelogramo para entrada y salida de datos, rectángulo para procesos y rombo para decisiones. A continuación se completó la tabla de prueba de escritorio con los datos de prueba de la consigna. Finalmente, la lógica se codificó en C++ y la ejecución real se comparó contra la traza manual: ambas salidas coincidieron en el cien por ciento de los casos.

Ejercicio 1 — El Sensor de Aprobación (diagrama en verde)

Un sistema académico necesita evaluar si un estudiante aprueba una asignatura técnica a partir de sus calificaciones. La solución combina tres conceptos: la constante, la entrada de datos por teclado y la estructura condicional. La constante se declara como const int NOTA_MINIMA = 6: a diferencia de una variable, su valor queda fijado en tiempo de compilación y no puede reasignarse durante la ejecución, lo que protege el umbral de aprobación de modificaciones accidentales. El programa solicita la nota de cada cuatrimestre (cin >> nota1 y cin >> nota2), calcula promedio = (nota1 + nota2) / 2 y resuelve el rombo de decisión con una estructura if/else: si la condición relacional promedio >= NOTA_MINIMA es verdadera muestra «APROBADO»; en caso contrario muestra «A INTENSIFICACIÓN». La Tabla 1 presenta la prueba de escritorio de los dos casos exigidos por la consigna.

Tabla 1
Prueba de escritorio del Ejercicio 1: el sensor de aprobación (constante NOTA_MINIMA = 6).

Caso	nota1	nota2	promedio = (nota1 + nota2) / 2	¿promedio >= NOTA_MINIMA?	Salida en pantalla
A	4	8	(4 + 8) / 2 = 6	6 >= 6 → Verdadero	APROBADO
B	3	5	(3 + 5) / 2 = 4	4 >= 6 → Falso	A INTENSIFICACIÓN

Ejercicio 2 — El Bucle del Contador de Vidas (diagrama en amarillo)

Este ejercicio modela el ciclo de vida de una entidad dentro de un motor de videojuego. La estructura repetitiva elegida es el bucle pre-test while (vidas > 0): antes de cada vuelta se evalúa la condición del rombo y solo si resulta verdadera se ejecuta el cuerpo del bucle. Dentro del cuerpo se muestra por pantalla «Jugador activo. Vidas restantes: » seguido del valor actual de la variable, y luego se simula el impacto de un obstáculo con el decremento vidas = vidas − 1. La flecha de retorno del diagrama de flujo vuelve exactamente a la evaluación de esa condición, formando el lazo del ciclo. Cuando la variable llega a 0, la comparación 0 > 0 es falsa, el bucle termina y el flujo continúa con

la salida «GAME OVER». La Tabla 2 traza el valor de la variable de control en cada iteración hasta la salida: 3 → 2 → 1 → 0.

Tabla 2
Prueba de escritorio del Ejercicio 2: el bucle del contador de vidas (vidas inicial = 3).

Iteración	¿vidas > 0? (rombo)	Salida por pantalla	vidas = vidas − 1
1.ª vuelta	3 > 0 → Verdadero	«Jugador activo. Vidas restantes: 3»	2
2.ª vuelta	2 > 0 → Verdadero	«Jugador activo. Vidas restantes: 2»	1
3.ª vuelta	1 > 0 → Verdadero	«Jugador activo. Vidas restantes: 1»	0
Salida del bucle	0 > 0 → Falso	«GAME OVER»	—

Ejercicio 3 — Registro de Jugador en Memoria (diagrama en naranja)

Un registro de jugador requiere una agrupación heterogénea de datos bajo una misma entidad conceptual. A diferencia del arreglo (colección homogénea, donde todos los elementos son del mismo tipo), la estructura struct permite modelar un objeto con campos de tipos distintos. El modelo diseñado posee tres campos: nickname (texto, string), nivel (entero, int) y esVIP (booleano, bool: solo admite Verdadero o Falso). En el flujo principal se instancia el objeto jugador1, se cargan sus atributos por teclado accediendo a cada campo con el operador punto (jugador1.nickname, jugador1.nivel, jugador1.esVIP) y se evalúa el rombo sobre el campo booleano. La Tabla 3 representa el bloque de memoria RAM que ocupa el objeto subdividido en sus tres campos, con los valores cargados en la prueba de escritorio de la consigna.

Tabla 3
Bloque de memoria RAM del objeto jugador1 (struct Jugador) con la prueba de escritorio de la consigna.

Campo	Tipo de dato	Valor cargado por teclado
nickname	texto (string)	"Rothbard"
nivel	entero (int)	5
esVIP	booleano (bool)	Verdadero

Evaluación del rombo sobre el campo booleano: esVIP == Verdadero → la salida del programa es «Bienvenido al Servidor VIP, Rothbard».

Ejercicio 4 — Control de Inventario con Arreglos (diagrama en rojo)

Un almacén estructurado necesita la verificación automática de stock crítico. El arreglo (vector) es una colección homogénea de cinco valores enteros predefinidos: stock = [12, 0, 5, 30, 0], almacenado en las posiciones con índices del 0 al 4, ya que en C++ los arreglos se indexan desde cero. La Tabla 4 muestra la casilla del vector dibujada en papel con sus índices. El contador i se inicializa en 0 y el bucle while (i < 5) recorre el vector: en cada vuelta se verifica el elemento stock[i] con la comparación == 0; si coincide, se emite «ALERTA: Sin stock en la posición » seguido del índice; finalmente se incrementa el

contador con i = i + 1 para avanzar a la casilla siguiente. La Tabla 5 presenta la prueba de escritorio completa con las cinco iteraciones, que detecta las alertas en las posiciones 1 y 4: las únicas casillas cuyo valor es 0.

Tabla 4
Casilla del vector stock con sus índices del 0 al 4.

índice i	**0**	**1**	**2**	**3**	**4**
stock[i]	12	0	5	30	0

Tabla 5
Prueba de escritorio del Ejercicio 4: recorrido completo del vector stock.

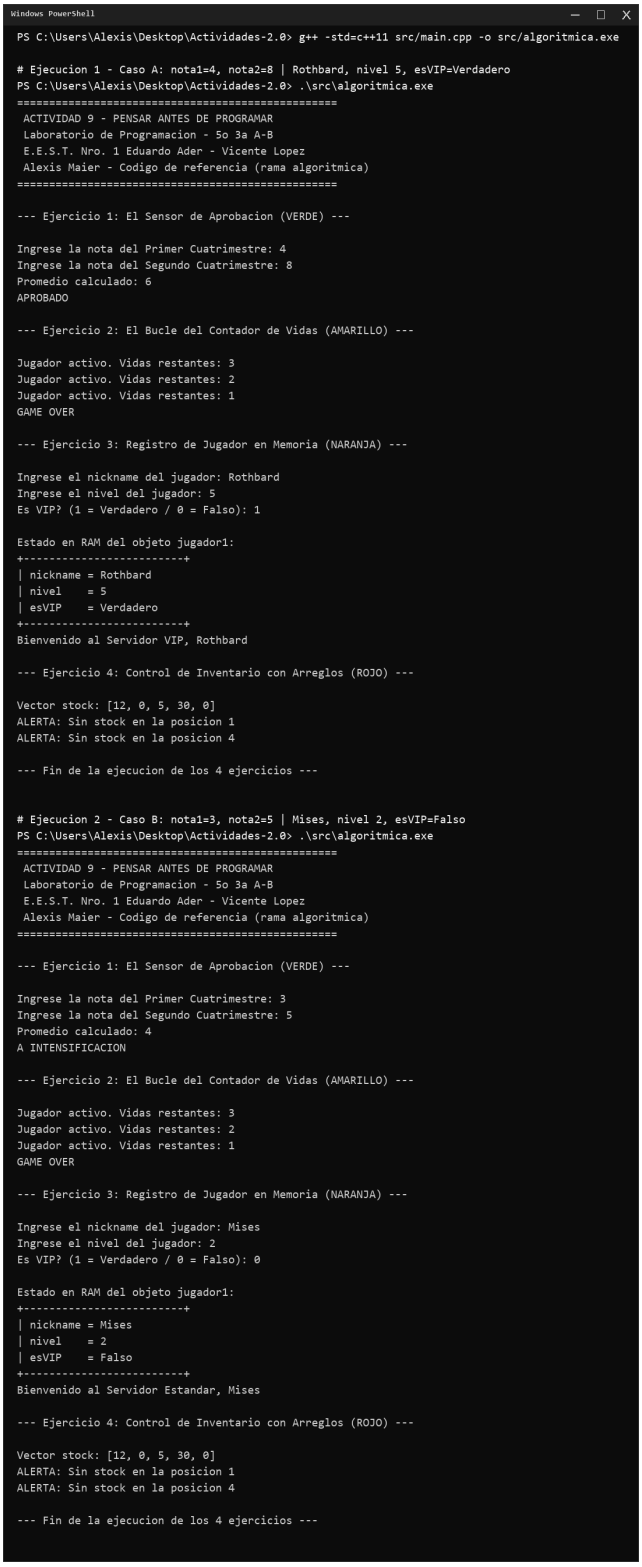
Iteración	**i**	**stock[i]**	**¿stock[i] == 0?**	**Salida por pantalla**
1.ª vuelta	0	12	No	(sin alerta)
2.ª vuelta	1	0	Sí	«ALERTA: Sin stock en la posición 1»
3.ª vuelta	2	5	No	(sin alerta)
4.ª vuelta	3	30	No	(sin alerta)
5.ª vuelta	4	0	Sí	«ALERTA: Sin stock en la posición 4»
Salida del bucle	5	—	5 < 5 → Falso	Fin del recorrido del vector

Código de referencia y matriz de ejecución

El código fuente que implementa los cuatro ejercicios se versiona en el archivo src/main.cpp de la rama algoritmica, organizado en una función por ejercicio más el flujo principal. La compilación se realizó con el compilador g++ (estándar C++11) desde la raíz del repositorio, con el comando g++ -std=c++11 src/main.cpp -o src/algoritmica.exe, sin errores ni advertencias. La Figura 1 muestra la matriz de ejecución en consola: la compilación y dos ejecuciones completas del programa con los datos de prueba. La salida real coincidió exactamente con las pruebas de escritorio realizadas a mano: el Caso A produjo promedio 6 y «APROBADO», el Caso B produjo promedio 4 y «A INTENSIFICACIÓN», el bucle de vidas mostró los mensajes con 3, 2 y 1 vidas antes del «GAME OVER», el jugador Rothbard (esVIP = Verdadero) recibió la bienvenida al Servidor VIP y el recorrido del vector emitió las alertas de las posiciones 1 y 4.

Figura 1

Matriz de ejecución del programa en consola: compilación con g++ y ejecución de los casos A y B.



Nota. Captura de la ventana de consola con la compilación y la ejecución completa del código de referencia con ambos casos de prueba de la consigna. Fuente: elaboración propia.

Pruebas de escritorio en la carpeta de la materia

Según la pauta de evaluación, las pruebas de escritorio a mano (fotografiadas o escaneadas desde la carpeta de la materia) representan el veinticinco por ciento de la nota. A continuación se reservan los espacios para las fotografías de las cuatro hojas manuscritas, que contienen el diagrama de flujo con su aplicación de color indicada y la cuadrícula de prueba de escritorio resuelta con los datos de la consigna.

Hoja 1 — Ejercicio 1: El Sensor de Aprobación — diagrama de flujo en VERDE + prueba de escritorio de los Casos A y B

[Pegar aquí la fotografía de la hoja manuscrita de la carpeta
Hoja 1: Ejercicio 1: El Sensor de Aprobación — diagrama de flujo en VERDE + prueba de escritorio de los Casos A y B]

Hoja 2 — Ejercicio 2: El Bucle del Contador de Vidas — diagrama de flujo en AMARILLO (con flecha de retorno) + tabla de iteraciones

[Pegar aquí la fotografía de la hoja manuscrita de la carpeta
Hoja 2: Ejercicio 2: El Bucle del Contador de Vidas — diagrama de flujo en AMARILLO (con flecha de retorno) + tabla de iteraciones]

Hoja 3 — Ejercicio 3: Registro de Jugador en Memoria — diagrama de flujo en NARANJA + bloque RAM subdividido en 3 campos (Rothbard, 5, Verdadero)

[Pegar aquí la fotografía de la hoja manuscrita de la carpeta
Hoja 3: Ejercicio 3: Registro de Jugador en Memoria — diagrama de flujo en NARANJA + bloque RAM subdividido en 3 campos (Rothbard, 5, Verdadero)]

Hoja 4 — Ejercicio 4: Control de Inventario — casilla del vector con índices 0–4 y diagrama de flujo en ROJO + tabla de las 5 iteraciones

[Pegar aquí la fotografía de la hoja manuscrita de la carpeta
Hoja 4: Ejercicio 4: Control de Inventario — casilla del vector con índices 0–4 y diagrama de flujo en ROJO + tabla de las 5 iteraciones]

Conclusión

La Actividad 9 cumplió su objetivo pedagógico de desacoplar el razonamiento lógico del lenguaje de programación. Resolver los cuatro ejercicios primero en papel obligó a razonar la solución con la simbología del diagrama de flujo y a anticipar el comportamiento del algoritmo antes de existir el programa, que es exactamente la competencia profesional que la actividad busca entrenar: cualquiera puede aprender la sintaxis de un lenguaje en unas semanas, pero descomponer un problema complejo en pasos lógicos claros es lo que distingue a un técnico en programación.

En lo técnico, la práctica integró los bloques fundamentos vistos durante el año: la diferencia entre constante y variable (el umbral NOTA_MINIMA quedó protegido gracias a const), la estructura repetitiva pre-test while con su variable de control y su condición de salida, la agrupación heterogénea struct para modelar la entidad Jugador en memoria RAM con campos de tipos distintos, y el recorrido indexado de un arreglo homogéneo desde el índice 0 hasta el 4. La prueba de escritorio demostró ser una herramienta de verificación estática: predijo el promedio 6 del Caso A, la secuencia de vidas 3 → 2 → 1 → 0, la bienvenida al Servidor VIP de Rothbard y las alertas de stock en las posiciones 1 y 4.

La verificación digital cerró el ciclo de trabajo: la ejecución del código de referencia en la rama algoritmica coincidió en el cien por ciento con lo trazado a mano, lo que valida tanto el razonamiento lógico como su implementación en C++. Versionar el trabajo de forma individual en Git, respetando la estructura de carpetas obligatoria con su licencia MIT, su .gitignore y su documentación, certifica el proceso completo y deja registro auditable de cada paso, tal como se espera de un flujo de trabajo profesional.

Referencias

GeeksforGeeks. (2024). Multidimensional arrays in C/C++. https://www.geeksforgeeks.org/multidimensional-arrays-c-cpp/

Git. (2024). Git documentation. https://git-scm.com/doc

Maier, A. (2026). Actividad 9: Práctica en papel — «Pensar antes de programar» [código fuente]. GitHub. https://github.com/alitomaier2010-netizen/Actividades-2.0/tree/algoritmica